

Neural Architecture Search for Joint Big/Little Dynamic Inference Networks

Kristoffer Petersen, Oliver Larsen, Oliver Svendsen, Aleksander Korsholm

University of Southern Denmark (SDU)

Supervisors: Emil Jorgensen, Francesco Daghero

Abstract—Dynamic inference can reduce average compute by using a small model for easy inputs and deferring harder inputs to a larger model. This work studies whether the two models in such a big/little cascade should be searched independently or jointly as one cascade. We propose a constrained NSGA-II co-search pipeline for CIFAR-10 that searches little-big CNN pairs under MCU-motivated memory and parameter constraints, optimizing proxy cascade accuracy and expected cascade FLOPs. The joint search is compared with an independent NAS baseline where little and big models are searched separately and combined afterwards. We also evaluate four retraining objectives: cross-entropy, cross-entropy with label smoothing, a cascade-aware co-training loss, and the same cascade-aware loss with label smoothing. Under a practical compute budget with 5-epoch proxy training and 50-epoch final retraining, joint co-search finds stronger final operating points than the independent baseline at the fixed confidence threshold $\tau = 0.70$. The cheaper cascade-aware variant reaches 79.69% accuracy at 1.67M MACs, while the highest-accuracy joint variant reaches 80.69% at 2.08M MACs; the independent baseline reaches 74.03% at 2.22M MACs. A Raspberry Pi Pico 2 deployment further shows lower measured latency and route-aware energy for the joint cascade than for the independent baseline. The results show that, within this constrained training setting, the accuracy-FLOPs trade-off is shaped not only by model accuracy, but also by threshold choice, exit ratio, routing error, and capacity allocation between the little and big models.

I. INTRODUCTION

Deep neural networks have become increasingly accurate, but their computational cost makes deployment difficult on resource-constrained devices. Edge and embedded systems often have strict limits on memory, latency, and energy, while still needing reliable predictions. A common way to reduce inference cost is dynamic inference: instead of spending the same computation on every input, the system adapts its computation to the difficulty of each sample. In a big/little cascade, a small model is evaluated first. If the small model is confident, the system exits early; otherwise, the sample is deferred to a larger model. Cascades can decide this route in different ways, for example using a learned gate, a margin-based rule, or a confidence threshold. This report uses a maximum-softmax confidence threshold because it adds no extra routing model, has negligible overhead, and is easy to evaluate under embedded constraints. Such a cascade can reduce average FLOPs substantially, but only if the little model is accurate enough on easy samples, the big model is strong enough on deferred samples, and the threshold routes examples correctly.

Designing such cascades is therefore not only a problem of finding two accurate models. The two models must work together under a shared routing rule and a shared deployment budget. If the little model is too small, it may rarely exit correctly. If the big model receives too much capacity, the cascade may become accurate but expensive. If the confidence threshold is too low, confident wrong exits can damage accuracy; if it is too high, most samples defer and the cascade loses its computational benefit. These trade-offs make cascade design a multi-objective problem involving accuracy, FLOPs, exit ratio, memory, and calibration.

Neural architecture search (NAS) is a natural tool for this setting because it can explore many architectural design choices under explicit constraints [1]. Prior work has studied NAS for efficient models, multi-objective architecture search, and dynamic inference, including early-exit networks and model cascades [2], [3], [4]. However, many cascade systems either design models independently or focus on placing exits inside a single backbone (Table I). This leaves open the question of whether a little and big model should be searched jointly as a pair, so that the search objective directly reflects cascade behavior.

This report studies that question using a CIFAR-10 big/little cascade under an edge-style deployment constraint. We compare a joint co-search method, where NSGA-II searches complete little-big pairs, with an independent baseline, where little and big models are searched separately and combined afterwards. Both methods use the same underlying CNN search space, the same proxy-training budget, and comparable resource constraints. The joint search optimizes cascade accuracy and average cascade FLOPs directly, while the independent baseline tests whether good standalone models are sufficient to produce good cascades.

Architecture search is only part of the problem. Since cascade routing depends on confidence, the final training objective may also matter. Cross-entropy (CE) optimizes classification accuracy, but it does not explicitly account for the fact that some samples should exit at the little model while others should defer. Confidence calibration and label smoothing are also relevant because miscalibrated confidence can cause incorrect early exits [5], [6], [7]. We therefore evaluate the selected joint architectures under four retraining settings: CE, CE with label smoothing, a cascade-aware loss, and the cascade-aware loss with label smoothing. This separates the architecture-search question from the training-objective

question.

The report is organized around three research questions:

- **RQ1:** Does joint NAS co-search of a little/big model pair produce a better final cascade accuracy versus average cascade FLOPs trade-off than independently searched and combined model pairs under matched search spaces and edge-memory constraints?
- **RQ2:** After architectures are selected by joint NAS, does cascade-aware co-training improve cascade accuracy, routing quality, or calibration compared with standard CE and CE with label smoothing?
- **RQ3:** How do threshold choice, exit ratio, and model-size allocation explain the observed accuracy/FLOPs trade-offs?

Taken together, these questions define the experimental framework studied in this report: constrained joint architecture search for NAS-driven big/little dynamic inference, a matched independent-search baseline, cascade-aware retraining, and threshold-sweep evaluation of input-adaptive routing. In this work, framework refers to the complete experimental pipeline for constrained joint cascade search, retraining, and threshold-based evaluation, rather than a general-purpose reusable software package. The implementation and experiment scripts are available in the project repository.¹ The final evaluation shows that the framework’s main advantage comes from designing the cascade as a coupled system: the joint variants improve the accuracy-FLOPs trade-off over the independently searched baseline at the fixed operating threshold. The cascade-aware loss is therefore treated as a retraining ablation rather than the sole contribution; it is useful for analyzing and tuning routing behaviour, while the C1-C4 variants expose the trade-off between cheaper early exits and higher-accuracy deferral.

The main contributions of this work are:

- a joint NSGA-II co-search setup for big/little CNN cascades under memory and parameter constraints;
- a cascade-aware co-training loss using soft routing probabilities to align training with threshold-based inference;
- an ablation study of CE, CE with label smoothing, cascade-aware loss, and cascade-aware loss with label smoothing; and
- an analysis showing that joint cascade design improves the observed accuracy/FLOPs trade-off, while threshold, exit ratio, routing errors, and capacity allocation explain the remaining differences between variants; and
- a Raspberry Pi Pico 2 deployment evaluation reporting on-device latency, route split, deployment-memory observations, and route-aware energy consumption.

The remainder of the report is structured as follows. Section II reviews dynamic inference, NAS for efficient models, and confidence calibration for cascade routing. Section III describes the cascade model, search space, joint and independent NAS procedures, and retraining losses. Section IV presents the experimental setup and evaluation metrics. Section V reports the results for the three research questions. Section VI

discusses the main findings, limitations, and future work, and Section VII concludes the report.

II. RELATED WORK

A. Dynamic Inference and Big/Little Cascading

Dynamic neural networks adapt their computation to each input instead of using a fixed inference path for all samples. Han et al. survey this area and group dynamic models into instance-wise, spatial-wise, and temporal-wise adaptation methods [8]. Big/little cascades are an instance-wise dynamic inference method: an inexpensive model handles easy samples, while difficult samples are routed to a more expensive model. Park et al. proposed an early big/little DNN system for low-power inference, showing that a small network can filter easy inputs before invoking a larger network [9]. This idea is attractive for embedded systems because the expected cost depends on the exit ratio, not only on the worst-case cost of the largest model.

Closely related work studies early-exit networks, where auxiliary classifiers are attached to intermediate layers of a single backbone. BranchyNet introduced side branches that allow confident samples to exit before reaching the final layer [10]. MSDNet extends this idea with multi-scale feature representations for anytime and budgeted classification [11]. Shallow-Deep Networks analyze “overthinking”, where a deep network can change an already-correct intermediate prediction into an incorrect final one, and use early exits to reduce unnecessary computation [12]. These methods demonstrate the value of adaptive computation, but they primarily modify a single backbone. Our work instead studies a cascade of two independent CNNs and asks how capacity should be allocated between the little and big models.

The routing rule is central to both early-exit networks and model cascades. Most systems use confidence thresholds: a sample exits if the intermediate or little model is sufficiently confident. This makes calibration important, because overconfident wrong predictions directly become wrong early exits. Learning to Cascade addresses this issue by calibrating confidence specifically for cascade systems, showing that standard calibration methods do not always optimize the accuracy-cost trade-off of a cascade [7]. This motivates our separate analysis of confidence, routing errors, and retraining losses.

B. NAS for Dynamic Inference

NAS automates architecture design by defining a search space, a search strategy, and a performance-estimation method [1]. It is particularly relevant for dynamic inference because accuracy and compute must be optimized jointly. Multi-objective evolutionary NAS is a common fit for this setting, since it can produce a Pareto front rather than a single scalarized solution. NSGA-II is a standard elitist multi-objective genetic algorithm [2], and NSGA-Net applies multi-objective evolutionary search to neural architectures [3]. We follow this general multi-objective view, but apply it to cascade pairs under explicit memory and parameter constraints.

¹Neural-Efficiency-Research-Unit-NERU/nas-dynamic-big-little-cascade

Several recent methods apply NAS to early-exit neural networks. EDANAS searches early-exit networks by adapting an Once-for-All style search space and jointly considering accuracy and MACs [13]. NACHOS extends this line toward hardware-constrained early-exit design, searching for networks that satisfy accuracy and MAC constraints at inference time [14]. These approaches are close in spirit because they use architecture search for adaptive inference. However, they focus on adding or optimizing exits within a single network. Our method searches two separate role-specific networks and optimizes their behavior as a cascade pair.

Other work focuses on learning the exit decision itself. JEI-DNN jointly learns the gating mechanism and intermediate inference modules, addressing the train-test mismatch that arises when confidence-threshold gates are trained separately from the prediction modules [15]. This line of work is complementary to ours: JEI-DNN improves learned gating inside dynamic neural networks, while our study keeps a simple threshold router and asks whether the architectures of the two cascade members should be searched jointly.

ENCAS is the closest related work on NAS for model cascades. It performs evolutionary neural cascade search across pretrained supernetworks to obtain a front of cascades trading accuracy against FLOPs [4]. Our work differs in two ways. First, we co-search the little and big models from a small role-specific CNN search space under a strict edge-memory budget, rather than composing subnetworks from pretrained supernetworks. Second, we explicitly compare joint co-search against an independently searched baseline and then study whether cascade-aware retraining improves the selected architectures.

C. Confidence Calibration for Cascade Routing

Modern neural networks are often miscalibrated: their predicted probabilities can be more confident than their empirical accuracy [5]. This is especially important for cascades because the confidence score is not only a diagnostic quantity; it controls computation. A poorly calibrated little model can exit on samples that should have been deferred to the big model. As a result, optimizing standalone classification accuracy is not necessarily enough to optimize cascade accuracy.

Label smoothing is one common training modification that can affect both accuracy and calibration. Muller et al. show that label smoothing changes the structure of learned representations and can improve generalization, although its effect depends on the task and downstream use [6]. Focal loss has also been studied as a calibration-aware alternative; Mukhoti et al. show that it can improve calibration in some settings [16]. These results motivate treating the final training objective as part of the cascade design, rather than evaluating only the searched architecture.

Cascade-specific calibration is more directly aligned with our problem. Learning to Cascade argues that calibration should be optimized for the accuracy-cost behavior of the whole cascade, not only for matching confidence to model accuracy [7]. Our cascade-aware loss follows the same motivation but is used as a retraining objective for a jointly

searched big/little pair. It introduces soft routing weights during training so that little-model and big-model losses are weighted according to the expected route of each sample. The goal is to improve cascade behavior without replacing the simple threshold rule used at inference.

D. Resource-Constrained NAS

Hardware-aware NAS extends architecture search with deployment constraints such as latency, memory, energy, FLOPs, or parameter count. This is important because the most accurate architecture under an unconstrained search is often not deployable on edge hardware. Surveys of hardware-aware NAS describe how search objectives can be coupled to device-specific measurements or analytical proxies [17]. NetAdapt iteratively adapts a pretrained network to meet platform constraints [18], while ProxylessNAS searches directly on the target task and hardware instead of relying only on proxy tasks [19]. MobileNetV3 combines platform-aware search with efficient mobile building blocks [20].

During NAS, our search uses analytical resource constraints rather than measured latency: deployment memory, parameter count, and average cascade FLOPs. This keeps architecture selection reproducible and independent of measurement noise from a specific hardware run. Measured Pico 2 latency is treated separately as a deployment evaluation after model selection. The memory constraint is applied to the whole cascade, not just to each model separately. This matters because a good cascade is a capacity-allocation problem: the little model must be large enough to exit correctly, while the big model must still have enough capacity to improve deferred samples.

E. Gap in Existing Work

Table I summarizes how the most relevant methods relate to this work. Existing work strongly supports the usefulness of dynamic inference, early exits, confidence-aware routing, and hardware-aware NAS. However, two gaps remain. First, most NAS methods for dynamic inference search early exits inside one backbone or assemble cascades from pretrained models; few study co-search of two independent role-specific architectures under a shared cascade objective. Second, the training objective for cascades is still not settled. A loss that improves routing may also harm branch accuracy, so it must be evaluated separately from the architecture search itself.

III. METHODOLOGY

This section describes the cascade inference system, the architecture search space, and the two design procedures compared in the experiments. The methodology separates two questions: RQ1 evaluates whether the two models should be searched jointly or independently, while RQ2 evaluates whether the final retraining loss improves the selected cascades.

TABLE I
COMPARISON OF RELATED DYNAMIC-INFERERENCE AND NAS METHODS.

Method	Main idea	Architecture setting	Relation to this work
BranchyNet [10]	Early exits with side classifiers	Single backbone	Motivates confidence-based early exit
MSDNet [11]	Multi-scale anytime/budgeted classification	Single backbone	Shows adaptive compute improves efficiency
Park et al. [9]	Big/little low-power inference	Two-model cascade	Foundational cascade design, not NAS co-search
EDANAS [13]	NAS for early-exit networks	Single early-exit network	Searches exits, not separate little/big models
NACHOS [14]	Hardware-constrained early-exit NAS	Single early-exit network	Adds hardware constraints to early-exit NAS
JEI-DNN [15]	Jointly learned exit and inference modules	Learned gating in dynamic network	Learns gating; does not co-search cascade pair
ENCAS [4]	Evolutionary search over model cascades	Pretrained supernetwork cascades	Closest cascade NAS; different search setting
Ours	Joint NSGA-II co-search of little/big CNNs	Two independent role-specific models	Compares joint vs. independent search and losses

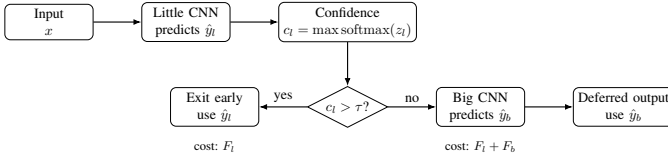


Fig. 1. Threshold-based routing in the big/little cascade. The little model is always evaluated. If its confidence is above the threshold τ , the cascade exits early; otherwise, the input is deferred to the big model.

A. Cascade Architecture and Routing

The cascade consists of two independent CNNs: a lightweight little model and a larger big model. For each input x , the little model is evaluated first. Its confidence is computed as the maximum softmax probability,

$$c_l = \max(\text{softmax}(z_l)),$$

where z_l are the little-model logits. If $c_l > \tau$, the cascade accepts the little model’s prediction and exits early. Otherwise, the input is deferred to the big model.

Figure 1 shows this threshold-based routing rule. Every input is processed by the little model, but only inputs below the confidence threshold τ are sent to the big model.

We report the exit ratio ρ , defined as the fraction of samples handled by the little model alone. The expected cascade FLOPs are then

$$F_{\text{cascade}} = F_l + (1 - \rho)F_b,$$

where F_l and F_b are the FLOPs of the little and big models. This metric captures the dynamic-inference cost: every sample pays for the little model, while only deferred samples pay for the big model. We also report routing error rate, which measures cases where the little model exits incorrectly although the big model would have predicted correctly.

TABLE II
CNN SEARCH SPACE USED FOR JOINT AND INDEPENDENT NAS.

Component	Little model	Big model
Blocks	1-3	2-5
Channels	{8, 12, 16, 24, 32}	{16, 24, 32, 48, 64}
Layers/block	{1, 2}	
Kernel size	{3, 5}	
Convolution	standard or depthwise separable	
Residual	enabled or disabled	

B. Search Space and Resource Constraints

Both models are sampled from the same CNN search space, but with role-specific channel ranges and depths. The little model uses the smaller role configuration, while the big model uses the larger one. Table II summarizes the searchable block counts, channel choices, and per-block operations used by both the joint and independent NAS experiments.

The search space is intentionally small and discrete. This keeps evolutionary search feasible under the available compute budget while still allowing meaningful variation in depth, width, convolution type, and residual connections. The little and big channel ranges are separated so that the little model is biased toward cheaper architectures, while the big model has enough capacity to serve as a fallback for harder samples. The block ranges follow the same role separation: the little model is limited to fewer stages, while the big model can use deeper feature extraction.

All candidates must satisfy deployment constraints motivated by embedded inference. The deployment-memory estimate includes both models’ weights, the input buffer, and the larger of the two peak activation footprints, since the cascade evaluates the models sequentially. Joint NAS enforces a 450 KiB deployment-memory cap, a 50K-100K combined-parameter window, role-level lower bounds of little $\geq 15\text{K}$ and big $\geq 40\text{K}$ parameters, role-level upper bounds of little $\leq 30\text{K}$ and big $\leq 85\text{K}$ parameters, the role-order constraint that the

big model must have at least as many parameters as the little model, and the structural rule that the big model must have more blocks than the little model. The independent baseline uses the same search space and role-order rules, but searches the roles separately with the same explicit parameter ranges: 15K-30K for little and 40K-85K for big.

The 450 KiB cap is chosen as a practical target for deployment on a 512 KiB MCU. It leaves some margin for runtime overhead, stack usage, framework code, and other buffers instead of assuming that the full 512 KiB can be used only for model storage and activations. This creates a tight embedded-memory setting where the search must make meaningful capacity trade-offs instead of simply increasing both models. The parameter lower bounds prevent the search from exploiting extremely small models that are cheap but not useful classifiers, while the upper bounds keep the two roles within the same capacity envelope for both search strategies. The big-model upper bound is higher because a 15K little model can still be paired with an 85K fallback without exceeding the 100K combined cap. The role-order constraints prevent the first-stage model from becoming larger than the fallback model. Together, the parameter and block-order rules enforce a clear separation between the cheap first-stage model and the more expressive fallback model.

The memory and FLOP constraints used during search are analytical estimates. Parameters and activations are counted as float32 values, not quantized weights. This keeps the search reproducible across execution environments, but it also means the NAS resource numbers should be interpreted as model-level deployment proxies. Direct Pico 2 latency is reported separately after the architectures have been selected and exported.

C. Joint Co-Search with NSGA-II

The joint NAS experiment searches little-big model pairs directly. Each candidate encodes both architectures and is first checked against the hard memory, parameter, and depth-order constraints. Infeasible candidates are rejected analytically and are not proxy-trained. Feasible candidates are proxy-trained for 5 epochs using the cascade-aware shared loss, then evaluated as a cascade at a fixed threshold $\tau = 0.70$. The co-search proxy loss uses $k = 20$, $\lambda_l = 0.75$, $\lambda_b = 0.05$, $\lambda_d = 0.01$, and $\lambda_w = 0.075$, with no label smoothing. The 5-epoch proxy budget is a compromise: it is long enough to separate clearly weak architectures from promising ones, but short enough to evaluate hundreds of candidates within the available compute budget.

The threshold is fixed during architecture search rather than optimized as part of the genotype. This avoids a shortcut where the search can improve proxy cascade accuracy by choosing a high threshold that defers almost every sample to the big model. Thresholds are instead swept after final retraining, where the accuracy-FLOPs trade-off can be analyzed explicitly. The value $\tau = 0.70$ was chosen as a practical proxy-search setting: it is low enough that the little model can exit

on some samples during proxy evaluation, but high enough that early exits still require meaningful confidence.

Infeasible architectures are filtered before proxy training using analytical parameter, memory, and block-order checks. The evolutionary initialization and offspring repair steps also attempt to sample or repair candidates into the feasible region. This prevents the search budget from being dominated by invalid architectures and makes the number of proxy-trained candidates more meaningful. For joint co-search, these checks include both pair-level constraints and role-level little/big parameter ranges before the candidate is proxy-trained.

NSGA-II is used because the search is naturally multi-objective: we want high cascade accuracy and low average cascade FLOPs under hard deployment constraints. Rather than choosing an arbitrary scalar weighting between accuracy and compute, NSGA-II produces a Pareto front of architecture pairs and makes the accuracy-compute trade-off explicit. It is also a practical choice for this study because it is well-established, straightforward to implement for mixed discrete architecture variables, and sufficient for answering whether joint pair search improves the observed accuracy-FLOPs trade-off. The two objectives are therefore to maximize proxy cascade accuracy and minimize expected cascade FLOPs. Candidates with exit ratios outside the desired routing window are softly penalized, so the search does not only find accurate pairs that almost always defer to the big model.

After the search, three candidates are selected for full retraining from the proxy Pareto region. The selector keeps only candidates whose expected cascade FLOPs are lower than the big model’s standalone FLOPs, $F_{\text{cascade}} < F_b$, so the selected pair represents a genuine compute-saving cascade rather than a slower two-stage system. Within this filtered region, the chosen candidates are the knee point of the proxy accuracy-FLOPs frontier and its nearest unique neighbours. This favors architectures around the point where additional compute gives diminishing proxy-accuracy gains, while avoiding duplicate genotypes. The selection rule therefore does not simply take the three highest proxy-accuracy candidates; it chooses representative candidates from the compute-saving knee region and leaves final performance claims to the fully retrained test results. The proxy frontier and selected candidates are shown in Figure 10 in Appendix A.

D. Independent NAS Baseline

The independent baseline is designed to test whether joint pair search provides value beyond searching two good models separately. In this baseline, little and big architectures are optimized in two separate NSGA-II searches. Each search optimizes standalone validation accuracy against parameter count within its role-specific budget. Parameter count is used at this stage because each model is searched before cascade routing exists, so expected cascade FLOPs cannot yet be computed. Generated role candidates are checked analytically against their role-specific parameter ranges before proxy training: little candidates must be within 15K-30K parameters, while big candidates must be within 40K-85K parameters.

The initial population samples only role-feasible candidates, and offspring outside the configured role range are repaired by resampling a feasible replacement before evaluation. The resulting role-specific Pareto candidates are then combined post hoc into cascade pairs. At this stage, the combiner enumerates all little-Pareto by big-Pareto pairings rather than taking only the top few models from each role. Pair-level feasibility is checked after combination, including the total parameter budget, deployment-memory cap, and the role-order rules that the big model must have at least as many parameters and more blocks than the little model. The final independent cascades are ranked and selected from these feasible pairs. This means that a high-accuracy 85K big model is not discarded simply because it cannot pair with a 30K little model; it can still be selected if it forms a valid cascade with a smaller little model.

The baseline is matched to the joint search where this is meaningful: it uses the same CNN search space, seed, proxy-training duration, threshold sweep, and deployment constraints. It is intentionally not identical in objective, because its purpose is to test whether standalone little and big models are sufficient, or whether searching the two roles together under a cascade objective adds value. The combined independent pairs are retrained and evaluated using the same threshold sweep as the joint candidates.

Figure 2 summarizes the two design pipelines. The joint pipeline evaluates little-big pairs during search, while the independent baseline delays cascade construction until after the separate role-specific searches have finished.

E. Cascade-Aware Loss

We introduce a cascade-aware retraining loss that keeps standard classification terms, but also reflects how the cascade is used at inference time. In this work, *co-training* means jointly optimizing the little and big models in the same cascade pair with a shared loss. It does not refer to semi-supervised co-training with separate data views. The key idea is to replace the hard routing decision in Figure 1 with a differentiable approximation during training. Figure 3 shows how the loss is constructed from the two model outputs and the soft routing probability.

First, we compute the little model’s confidence:

$$c_l = \max(\text{softmax}(z_l)),$$

where z_l are the little-model logits. Instead of directly applying the hard rule $c_l > \tau$, we define a soft exit probability:

$$p_{\text{exit}} = \sigma(k(c_l - \tau)),$$

where τ is the routing threshold, k controls how sharp the soft decision is, and σ is the sigmoid function. If the little model is confident, p_{exit} becomes close to 1. If it is uncertain, p_{exit} becomes close to 0.

As illustrated in Figure 3, a high-confidence little prediction is treated as a likely early exit, so the little-model CE term is emphasized. A low-confidence little prediction is treated as a likely deferral, so the big-model CE term is emphasized.

The routing terms are small regularizers rather than the main training signal: they encourage the cascade to avoid deferring every sample, while also discouraging confident wrong little-model exits.

The full cascade-aware loss is:

$$\begin{aligned} \mathcal{L}_{\text{cascade}} = & (p_{\text{exit}} + \lambda_l)\text{CE}(z_l, y) \\ & + (1 - p_{\text{exit}} + \lambda_b)\text{CE}(z_b, y) \\ & + \lambda_d(1 - p_{\text{exit}}) \\ & + \lambda_w p_{\text{exit}} \mathbf{1}[\hat{y}_l \neq y]. \end{aligned}$$

The first term trains the little model. It is weighted by the soft exit probability because the little model is responsible for samples likely to exit early. The auxiliary weight λ_l ensures that the little model still receives a stable classification signal even for samples that are likely to be deferred.

The second term trains the big model. It is weighted by $1 - p_{\text{exit}}$, so the big model is trained more strongly on samples that the little model is likely to defer. The optional auxiliary weight λ_b keeps a small constant big-model training signal even for samples that are likely to exit early.

The third term, $\lambda_d(1 - p_{\text{exit}})$, is a small defer cost. It discourages the system from sending every sample to the big model, which would increase accuracy but destroy the FLOPs benefit of the cascade. This weight is kept small because the model should not be forced to exit when the little model is genuinely uncertain; if the penalty is too large, it can dominate the classification objective and encourage harmful early exits.

The final term, $\lambda_w p_{\text{exit}} \mathbf{1}[\hat{y}_l \neq y]$, penalizes confident wrong exits. This is the most cascade-specific part: it targets the harmful case where the little model would accept a sample even though its prediction is wrong. Like the defer cost, it is used as a regularizer so that the classification losses remain the dominant optimization signal; overly large values could reduce branch accuracy or calibration instead of improving cascade behavior.

Overall, the loss tries to balance three goals: keep the little model accurate, keep the big model useful for deferred samples, and encourage early exits only when they are likely to be correct. In the final C3 and C4 experiments we use routing sharpness $k = 20$, $\lambda_l = 0.75$, $\lambda_b = 0.05$, $\lambda_d = 0.0125$, and $\lambda_w = 0.075$. C3 uses no label smoothing, while C4 uses label smoothing of 0.1.

F. Retraining Loss Ablations

For RQ2, the selected joint-NAS architectures are retrained from scratch under the four loss settings summarized in Table III. This isolates whether improvements come from standard cross-entropy, label smoothing, the proposed cascade-aware loss, or the combination of cascade-aware loss and smoothing. CE is used as the default baseline because it is the common supervised classification objective for multi-class neural networks [?] and is widely used as a baseline in CIFAR-style image-classification experiments [?]. It directly optimizes branch prediction accuracy without adding cascade-specific assumptions. Label smoothing is included because cascade routing depends on confidence, and smoothing is a simple

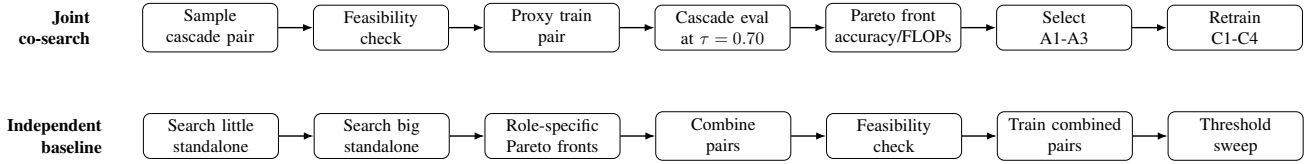


Fig. 2. Comparison of the joint co-search pipeline and the independent NAS baseline. Joint NAS searches full cascade pairs directly, whereas the independent baseline searches the two roles separately and only later assembles cascade pairs.

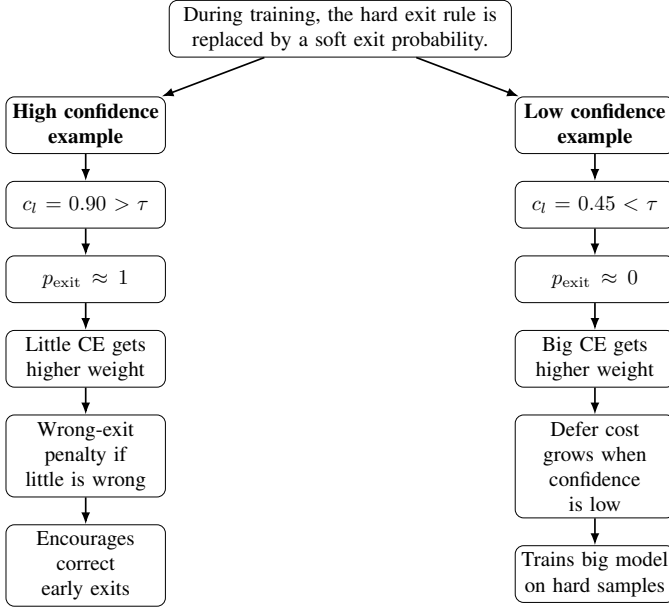


Fig. 3. Behavioral view of the cascade-aware loss. High-confidence samples are treated as likely early exits, while low-confidence samples are treated as likely deferrals. The soft exit probability shifts the learning emphasis between the little and big models.

calibration-oriented baseline for testing whether less overconfident branch predictions improve routing. The comparison is performed after architecture selection, so RQ2 evaluates the retraining objective rather than the NAS search itself.

TABLE III
RETRAINING LOSS ABLATIONS.

Condition	Retraining loss
C1	Cross-entropy (CE)
C2	CE + label smoothing
C3	Cascade-aware loss
C4	Cascade-aware loss + label smoothing

IV. EXPERIMENTAL SETUP

A. Datasets

All experiments use CIFAR-10 [21]. We use the official CIFAR-10 training split for training and validation, with a deterministic 45K/5K split: the first 5K images are used for validation and the remaining 45K images are used for training. The official 10K CIFAR-10 test split is kept separate and is

used only for final threshold-sweep evaluation after retraining. CIFAR-10 is small enough to support repeated NAS proxy training and ablation runs, while still being non-trivial for confidence-based routing and calibration analysis.

The training split uses random crop with padding 4, random horizontal flipping, conversion to tensor, and CIFAR-10 normalization. The validation and test splits use only tensor conversion and the same normalization. The validation split is used during proxy NAS evaluation and joint-retraining checkpoint selection. The test split is not used for architecture selection.

B. Baselines

RQ1 compares two architecture-design procedures. The first is joint co-search, where NSGA-II searches complete little-big cascade pairs and evaluates them directly as cascades. The second is an independent NAS baseline, where the little and big architectures are searched separately and then combined after the two searches. Both approaches use the CNN search space in Table II, the same random seed, the same proxy-training duration, and the same final threshold sweep.

This comparison should be interpreted as an end-to-end cascade-design comparison rather than a perfectly isolated test of architecture search alone. The joint pipeline searches complete cascade pairs and uses cascade-aware pair training during proxy evaluation, while the independent baseline searches the two roles as standalone classifiers and trains the combined models independently with CE. Therefore, any RQ1 improvement reflects the combined effect of joint pair selection, cascade-aware proxy training, and matched resource constraints.

The joint search uses a population size of 20 for 20 generations. The independent baseline uses a population size of 20 for 10 generations for each role-specific search. Since it runs one little-model search and one big-model search, this gives the independent baseline approximately the same number of proxy-trained candidates as the joint search. This matches the proxy-evaluation budget more closely than the evolutionary depth per role: joint NAS evolves complete little-big pairs for 20 generations, while each independent role is evolved for 10 generations because the two searches are run sequentially and were substantially more time-consuming in practice. During search, joint candidates are evaluated at the fixed routing threshold $\tau = 0.70$. Independent candidates are searched as standalone classifiers using CE and optimized for validation accuracy against parameter count. Parameter count is used here because the models are searched before a cascade

TABLE IV
TRAINING AND EVALUATION SCHEDULE USED IN THE EXPERIMENTS.

Stage	Setting	Value
Joint NAS	Population / generations	20 / 20
Independent NAS	Population / generations	20 / 10 per role
Proxy training	Epochs / batch size	5 / 128
Proxy training	Optimizer	SGD, lr 0.05
Joint C1-C4 retraining	Epochs / batch size	50 / 128
Joint C1-C4 retraining	Optimizer	SGD, lr 0.1
Joint C1-C4 retraining	Scheduler	Cosine annealing
Independent final training	Epochs / learning rate	50 / 0.05
All final training	Momentum / weight decay	$0.9 / 5 \cdot 10^{-4}$
Threshold sweep	Thresholds	0.50, 0.60, 0.70, 0.80, 0.90, 0.95

route exists, so expected cascade FLOPs are not yet defined. After the two role-specific searches finish, the independent baseline combines the resulting Pareto fronts by enumerating all little-big pairs, rejecting pairs that violate the combined parameter budget, deployment-memory budget, or role-order rules, and selecting the highest-scored feasible cascade pairs. The selected little and big architectures are then trained as separate CE classifiers for 50 epochs; no cascade-aware shared loss is applied to the independent baseline. They are only coupled during final evaluation, where the little model’s confidence threshold determines whether an input exits early or defers to the big model. After combination, the independent cascades are evaluated with the same threshold sweep and cascade-FLOPs metric as the joint method. Some RQ1 plots use the C4 joint retrain directory to show the high-accuracy end of the joint ablation against the independently searched cascades, while the separate C1-C4 comparison in RQ2 reports how the final retraining loss changes accuracy, routing, and expected FLOPs for the same selected joint architectures.

For RQ2, the selected joint-NAS architectures are retrained under the four loss conditions in Table III. Each condition retrains the same three selected architectures, so the ablation compares retraining objectives rather than different architecture selections. Retraining starts from scratch rather than continuing from proxy-trained weights. This separates the short proxy phase, which is used only to choose architectures, from the longer final training phase used for reported results. The 50-epoch retraining budget is long enough to obtain reasonably stable comparisons, with smaller improvements expected in later epochs; a longer schedule such as 100 epochs would likely be more precise, but was not used because of the total compute cost of retraining multiple architectures under multiple loss settings.

C. Evaluation Metrics

The primary metrics are cascade accuracy and expected cascade FLOPs. Cascade accuracy is the accuracy after applying the threshold router: samples above the threshold use the little prediction, while deferred samples use the big prediction. Expected cascade FLOPs are computed analytically as $F_l + (1 - \rho)F_b$, where ρ is the measured exit ratio.

We also report exit ratio, little-model accuracy, big-model accuracy, parameter count, and estimated deployment memory. Deployment memory is estimated in float32 and includes both models’ weights, the input buffer, and the larger peak activation footprint of the two sequential paths. FLOPs are reported as MACs and are estimated analytically from the selected architectures. These analytical metrics are used for NAS selection and the main accuracy-compute comparisons; measured Pico 2 latency is reported separately in the deployment evaluation.

For routing and calibration analysis, we report little-model ECE and routing error rate. ECE is computed with 15 confidence bins using the little model’s maximum softmax confidence. Routing error rate is the fraction of test samples where the cascade exits at the little model, the little prediction is wrong, and the big model would have been correct. This captures the specific failure mode caused by a confident wrong early exit.

Proxy-search results and final retrained results are treated separately. A proxy result is a validation metric measured after short 5-epoch training and is used only for architecture selection. Proxy Pareto fronts therefore summarize which candidates looked promising under the cheap search budget. Final figures and comparisons use retrained checkpoints and test-set threshold sweeps, so final claims are not based on the proxy-search validation scores.

D. Implementation Details

All experiments are implemented in PyTorch and executed through a scripted pipeline that runs joint NAS, independent NAS, retraining, metric aggregation, and plot generation. CUDA is used when available, and CPU execution is used otherwise.

All CIFAR-10 experiments use seed 42. The implementation seeds Python, NumPy, PyTorch, CUDA when available, NSGA-II, feasible sampling and repair, and DataLoader shuffling. Search candidates are proxy-trained for 5 epochs, while final selected architectures are retrained from scratch for 50 epochs.

Experiment outputs are stored as CSV files, including search logs, Pareto fronts, selected architectures, training curves, and per-threshold test sweeps. All plots and analysis are generated from these saved outputs, so the reported figures can be reproduced without re-running model training.

E. Pico 2 Deployment Test Setup

After final retraining, the selected cascades were exported to ONNX and then to float32 TensorFlow Lite models for TensorFlow Lite Micro. The exported little and big models

were converted into C arrays and compiled into Raspberry Pi Pico 2 firmware. The Pico firmware implements the same threshold router used in the offline evaluation: each image is first evaluated by the little model, the maximum softmax confidence is compared with $\tau = 0.70$, and only low-confidence samples are deferred to the big model.

Two Pico experiments were run. First, the normal latency and accuracy test used a USB serial protocol in which the host streamed CIFAR-10 test images to the Pico one sample at a time. This allowed the full 10,000-image CIFAR-10 test set to be evaluated without storing the entire dataset in Pico flash. For each sample, the Pico reported the predicted class, route decision, correctness, little-model time, big-model time when used, total cascade time, and analytical FLOP counts for the selected path. This is the source of the deployment latency and route-split results reported in Table VII.

Second, a route-aware energy test was run on a 1,000-image embedded CIFAR-10 subset. This version stores the test subset in Pico flash and runs without USB power during measurement. Figure 4 shows the measurement circuit. An Elegoo Uno R3 powers the Pico 2 through a $10\ \Omega$ high-side shunt connected to VSYS. The Uno samples the voltage before and after the shunt to estimate current and power, while Pico GP18 drives an inference trigger connected to Uno D2. In the route-aware build, Pico GP19 also indicates the route decision to Uno D3, with the high state representing deferred big-model execution. This setup separates idle time from the active inference window and records whether each measured inference used the little path or the deferred path.

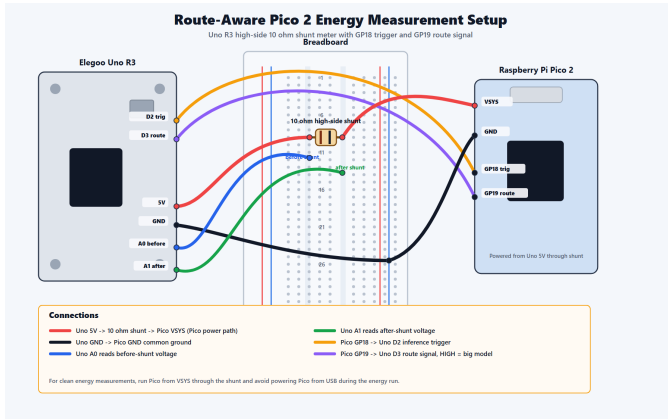


Fig. 4. Route-aware Pico 2 energy measurement setup using a $10\ \Omega$ high-side shunt, GP18 inference trigger, and GP19 route signal.

V. RESULTS

All reported results in this section use the final retrained checkpoints and the CIFAR-10 test split. Unless otherwise stated, results are reported at the fixed operating threshold $\tau = 0.70$, matching the main comparison used in the generated figures.

TABLE V

RQ1 TEST-SET COMPARISON AT $\tau = 0.70$. VALUES ARE AVERAGED ACROSS THE SELECTED RETRAINED CASCADES. C1-C4 SHARE THE SAME CO-SEARCHED ARCHITECTURES BUT DIFFER IN RETRAINING LOSS, WHILE THE INDEPENDENT BASELINE IS SEARCHED AND COMBINED SEPARATELY.

Method	Acc. (%)	Exit ratio	FLOPs (M MACs)	ECE
Joint NAS C1	79.64	0.55	1.69	0.0104
Joint NAS C2	80.33	0.43	2.01	0.0811
Joint NAS C3	79.69	0.55	1.67	0.0104
Joint NAS C4	80.69	0.40	2.08	0.0850
Independent NAS	74.03	0.57	2.22	0.0508

A. RQ1: Joint Co-Search vs. Independent NAS

Table V summarizes the final joint and independent cascade results. The joint co-search pipeline occupies a stronger accuracy-FLOPs region than the independent baseline. C3 is the cheaper cascade-aware variant, reaching 79.69% mean test accuracy at 1.67M average cascade MACs, while C4 reaches the highest mean accuracy, 80.69%, at a higher 2.08M average cascade MACs. The independent baseline reaches 74.03% at 2.22M average cascade MACs. Thus, at the fixed threshold used for the main comparison, the joint variants provide substantially higher accuracy, and the cheaper C3 operating point also uses less expected compute than the independent baseline.

The individual C4 cascades range from 79.77% to 81.71% accuracy at $\tau = 0.70$, while the independent cascades range from 72.67% to 75.96%. C3 gives a lower-compute joint operating point with similar accuracy to C1, whereas C4 gives the high-accuracy, high-deferral end of the ablation. This indicates that the joint pipeline is stronger than the independent baseline, but that the final operating point still depends on the retraining objective. The high-accuracy joint comparison is visualized in Figure 5.

B. RQ2: CE vs. Cascade-Aware Retraining

Table VI reports the retraining-loss ablation for the same three joint-NAS architectures. C1 and C3 compare CE against the cascade-aware loss without label smoothing. Their average cascade accuracies are close, but C3 slightly improves over C1 while also reducing expected FLOPs: C1 obtains 79.64% at 1.69M MACs, while C3 obtains 79.69% at 1.67M MACs. This is not a large standalone win, but it shows that the cascade-aware loss can move the operating point in the intended direction.

The smoothed variants, C2 and C4, achieve the highest average cascade accuracies. They also have substantially lower routing error rates than C1 and C3, but this comes with lower exit ratios and therefore higher expected cascade FLOPs. The results therefore show an accuracy-compute trade-off inside the ablation itself: C1 and C3 exit more often and are cheaper, while C2 and C4 defer more samples and obtain higher accuracy. C4 is therefore the higher-accuracy, higher-compute variant, whereas C3 is the cheaper cascade-aware alternative. These trends are shown in Figure 6.

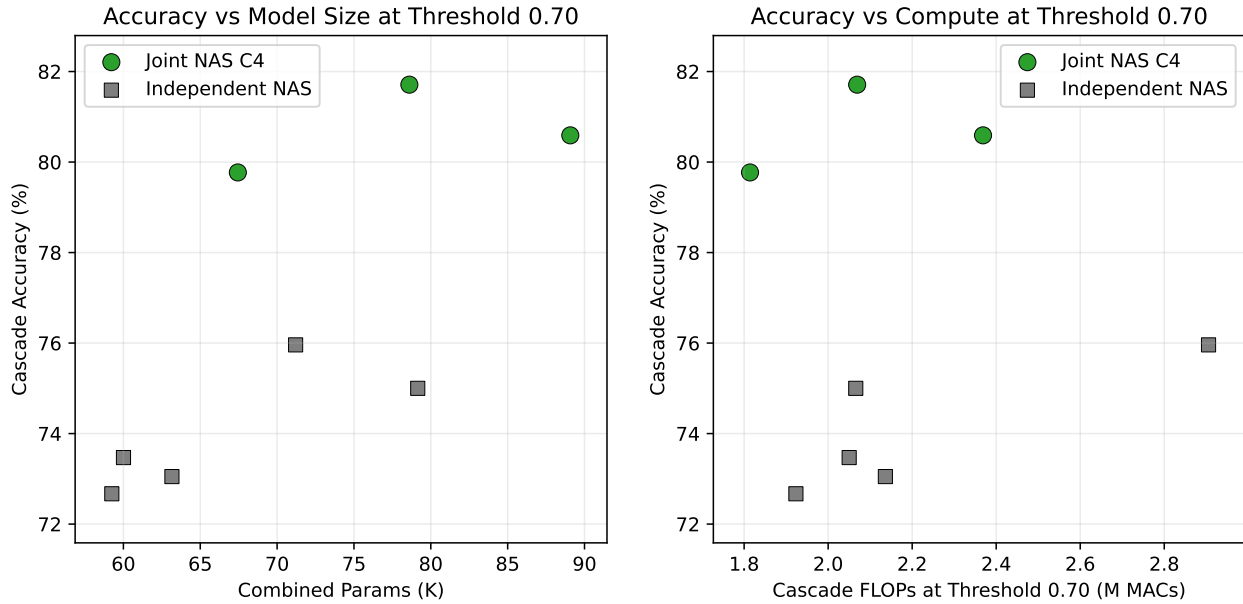


Fig. 5. Final retrained high-accuracy joint cascades compared with independently searched cascades at the fixed operating threshold $\tau = 0.70$. The plot uses the C4 retraining condition to show the high-accuracy end of the joint ablation, not to define C4 as the single canonical joint result. The left panel compares combined parameter count against cascade accuracy, while the right panel compares expected cascade FLOPs against cascade accuracy.

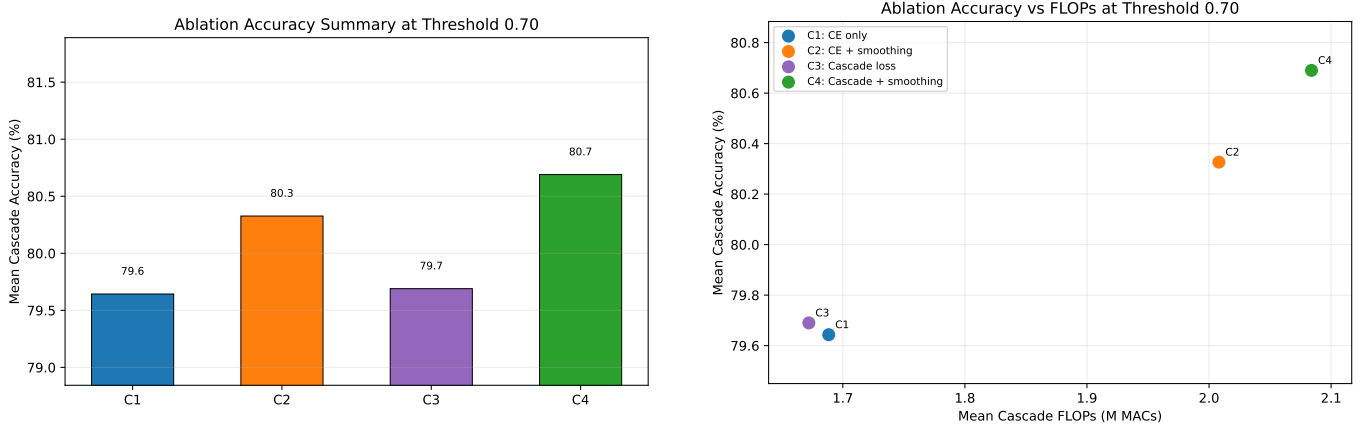


Fig. 6. Retraining-loss ablation for the three selected joint architectures at $\tau = 0.70$. C1 uses CE, C2 uses CE with label smoothing, C3 uses the cascade-aware loss, and C4 combines cascade-aware loss with label smoothing. The ablation shows that C3 slightly improves the CE operating point, while the strongest average accuracy and lowest routing error are associated with the smoothed variants.

TABLE VI

RQ2 RETRAINING-LOSS ABLATION AT $\tau = 0.70$. VALUES ARE AVERAGED OVER THE SAME THREE SELECTED JOINT ARCHITECTURES.

Variant	Acc. (%)	Exit ratio	FLOPs (M MACs)	Route err.
C1: CE	79.64	0.55	1.69	0.0231
C2: CE + smoothing	80.33	0.43	2.01	0.0093
C3: Cascade loss	79.69	0.55	1.67	0.0242
C4: Cascade + smoothing	80.69	0.40	2.08	0.0083

C. RQ3: Threshold, Exit Ratio, and Routing

The threshold-based cascade does not depend only on the standalone accuracy of the little and big models. It also depends on how confidence separates safe little-model exits from samples that should defer to the big model. This is visible in the routing metrics. At $\tau = 0.70$, C4 has a routing error rate of 0.0083, while C1 and C3 have routing error rates of 0.0231 and 0.0242, respectively. Thus, the highest-accuracy variants also make fewer harmful early exit decisions.

However, ECE and routing error do not move in the same direction. C1 and C3 have lower little-model ECE than C2 and C4, but they also have higher routing error. This means that global calibration, as measured by ECE, is not sufficient to

explain cascade quality. What matters for the threshold router is whether high-confidence little predictions are correct on the samples that exit early. Figure 7 illustrates this relationship between confidence quality, routing error, exit ratio, and cascade accuracy.

In summary, RQ3 shows that the accuracy-FLOPs trade-off is explained by the interaction between threshold, exit quality, and capacity allocation. The threshold controls how aggressively the cascade exits; the exit ratio determines how often the big model is avoided; routing error determines whether those exits are safe; and the little model’s FLOPs matter because they are paid for every input. The joint search performs better because it optimizes these factors as a coupled cascade, whereas the independent baseline can produce high exit ratios without equally strong routing quality or capacity allocation.

D. Pico 2 Deployment Results

To check whether the analytical cascade metrics transfer to an embedded target, the selected Arch2 C1–C4 cascades were exported from PyTorch checkpoints to ONNX and float32 TensorFlow Lite Micro models, then compiled into Raspberry Pi Pico 2 firmware. The independent NAS genotype 4 baseline was exported through the same path. The normal deployment test streamed the full CIFAR-10 test set over USB serial, so each operating point was evaluated on 10,000 images. All Pico runs used the same confidence threshold as the main comparison, $\tau = 0.70$, with temperature $T = 1.0$.

TABLE VII

PICO 2 SERIAL CIFAR-10 DEPLOYMENT RESULTS AT $\tau = 0.70$. THE LITTLE-PATH LATENCY IS THE END-TO-END LATENCY FOR SAMPLES THAT EXIT AFTER THE LITTLE MODEL. THE DEFERRED-PATH LATENCY INCLUDES BOTH THE LITTLE MODEL AND THE BIG MODEL.

Cascade	Acc.	Exit	Defer	Little ms	Defer ms	Avg ms
Arch2 C1	78.97	42.86	57.14	40.34	204.19	133.97
Arch2 C2	79.96	32.66	67.34	40.36	204.21	150.79
Arch2 C3	78.94	42.64	57.36	40.34	204.20	134.38
Arch2 C4	80.46	27.60	72.40	40.37	204.22	159.00
Independent 4	73.47	58.99	41.01	131.93	848.45	425.77

The Arch2 variants have almost identical per-path latency because they share the same exported architecture and differ mainly in trained weights and routing behaviour. Their little path takes about 40.3 ms, while a deferred sample takes about 204.2 ms end-to-end. The average latency therefore follows the observed route split: C1 and C3 are the fastest Arch2 variants because they exit on about 43% of the test set, whereas C4 reaches the highest Pico accuracy but defers 72.4% of samples and is correspondingly slower. The independent baseline exits more often than any Arch2 variant, but its little and deferred paths are much slower, giving a much higher average latency despite its higher exit ratio.

The deployment also exposed a memory distinction that is not visible from accuracy and FLOPs alone. The Arch2 firmware used separate TensorFlow Lite Micro arenas of 128 KiB for the little model and 256 KiB for the big model,

while the independent firmware required arena tuning and was finally tested with a 96 KiB little arena and a 352 KiB big arena. Additional independent energy builds with 160/288, 192/256, 224/224, and 256/192 KiB arena splits were produced to diagnose feasible allocations. This suggests that practical deployment feasibility depends not only on model weights and analytical FLOPs, but also on activation memory and TensorFlow Lite Micro arena requirements.

E. Energy Consumption

In addition to latency, a smaller route-aware energy experiment was run to measure whether the deployed cascade behaviour translates into lower energy use. Unlike the normal serial test, this energy run used a 1,000-image CIFAR-10 subset embedded directly in Pico flash. This avoided USB streaming during the measurement window and allowed the Pico to be powered through the Uno/shunt path shown in Figure 4. The Pico raised GP18 only during active cascade inference, so the Uno measurement excluded most idle time. The route-aware build also used GP19 to indicate whether a sample deferred to the big model.

TABLE VIII

ROUTE-AWARE PICO 2 ENERGY MEASUREMENT ON A 1,000-SAMPLE EMBEDDED CIFAR-10 SUBSET. THE JOINT ROW CORRESPONDS TO THE EXPORTED ARCH2 CO-SEARCHED CASCADE USED FOR THE FINAL PICO COMPARISON.

Cascade	Time (s)	Energy (J)	J/sample	Power (W)	Exit	Defer
Independent 4	428.7	44.0	0.044	0.099	58.5	41.5
Joint Arch2	330.1	34.3	0.034	0.105	53.0	47.0

The joint Arch2 cascade used 34.3 J over the embedded subset, compared with 44.0 J for the independent baseline. This is a 22.0% reduction in total energy and approximately the same reduction per image. Runtime also decreased from 428.7 s to 330.1 s, a 23.0% reduction. The average measured power is similar between the two runs, and is slightly higher for the joint cascade, so the energy saving is mainly explained by the shorter active inference time rather than by a lower instantaneous power draw.

The route counts make this result especially useful. In the energy run, the joint cascade actually deferred slightly more samples than the independent baseline, but still consumed less total energy. This matches the latency result in Table VII: the joint Arch2 paths are substantially faster, so the energy benefit does not come only from exiting more often. It comes from the deployed cost of each path. The independent baseline has a high exit ratio, but its little path and deferred path are both expensive on the Pico 2. This reinforces the conclusion that exit ratio, analytical FLOPs, and deployment memory must be interpreted together when judging whether a cascade is actually efficient on hardware.

VI. DISCUSSION

This section interprets the results in relation to the three research questions. The discussion focuses on what the experiments show about joint cascade design, retraining losses,

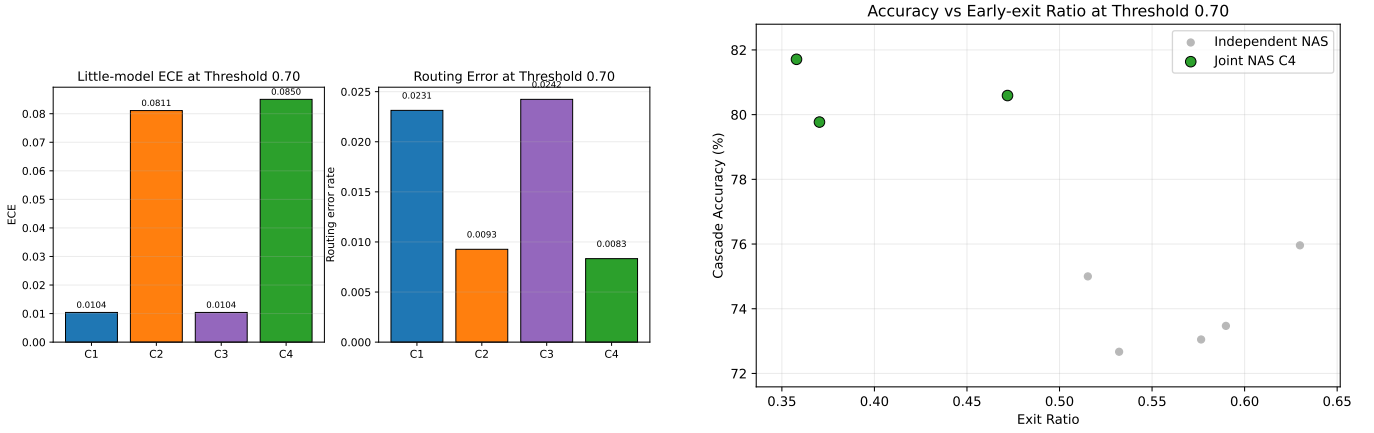


Fig. 7. Routing and confidence behaviour at $\tau = 0.70$. The left panel shows little-model ECE and routing error for the C1-C4 retraining variants, while the right panel relates cascade accuracy to early-exit ratio for joint and independent cascades. These metrics show why exit ratio must be interpreted together with confidence quality and accuracy.

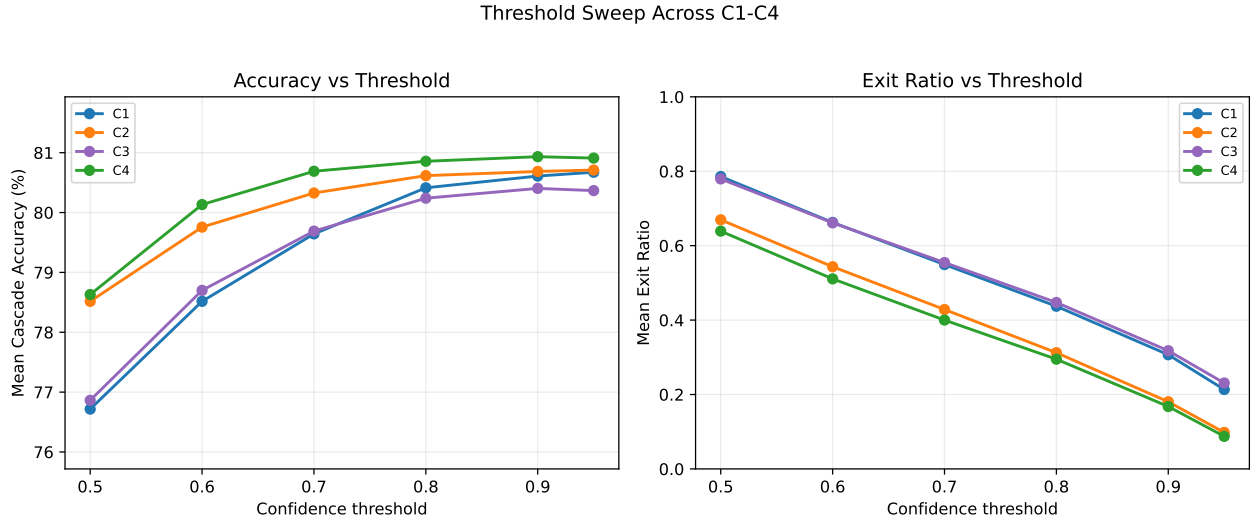


Fig. 8. Threshold sweep for the C1-C4 retraining variants. Increasing the confidence threshold reduces the early-exit ratio and generally improves cascade accuracy by deferring more samples to the big model. Lower thresholds produce cheaper, more exit-heavy cascades, while higher thresholds approach big-model behaviour. The cascade-aware variants, C3 and C4, were trained with the same threshold through a soft routing term, so their sweep also shows how that training objective transfers when the inference threshold is moved away from $\tau = 0.70$.

and confidence-based routing, and then summarizes the main limitations of the study.

A. Joint Cascade Design

The results support the idea that a big/little cascade should be designed as a coupled system rather than as two unrelated classifiers. The joint NAS pipeline selects architectures using cascade accuracy, expected cascade FLOPs, exit ratio, and deployment constraints at the same time. In contrast, the independent baseline first optimizes little and big models as standalone classifiers and only later combines them into cascade pairs. The RQ1 comparison therefore evaluates an end-to-end cascade-design pipeline. It does not isolate architecture search from joint retraining, but it does test the full procedure that would be used to design a deployed cascade.

At the fixed operating threshold $\tau = 0.70$, the co-searched cascades achieve substantially higher cascade accuracy than the independent baseline. The exact accuracy-compute point depends on the retraining loss: C3 is the cheaper cascade-aware point, while C4 obtains higher accuracy by deferring more samples and therefore using more FLOPs. This is the strongest result in the study: the joint search does not merely find a more accurate but more expensive cascade; it produces a set of stronger operating points under the same memory envelope. The independent baseline still exits more often, but its component models are weaker and its routing decisions produce lower final cascade accuracy.

Figure 5 shows this separation for the high-accuracy C4 setting. It should be read together with the C1-C4 ablation: C4 demonstrates the upper-accuracy joint setting, while C3

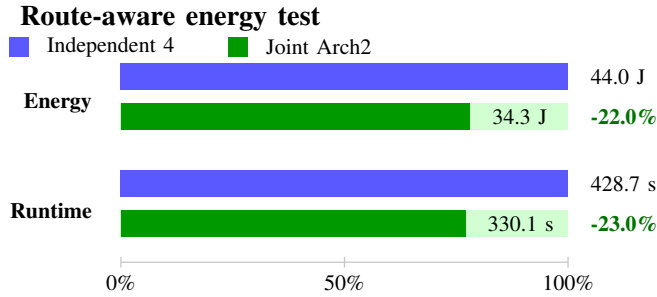


Fig. 9. Normalized route-aware Pico 2 energy-test comparison. Independent 4 is shown as the 100% baseline; the joint Arch2 cascade reduces total energy by 22.0% and runtime by 23.0% on the 1,000-sample embedded subset.

demonstrates the cheaper cascade-aware setting. Additional diagnostic plots in Appendix A break down little, big, and expected cascade FLOPs, showing how the selected architecture sizes and exit ratios combine to determine the final compute cost.

The FLOPs breakdown is important because the little model is evaluated for every input. In the independent baseline, the little model is selected as a standalone classifier under a parameter budget, so it can still be expensive in MACs if it uses costly convolutions on high-resolution feature maps. Parameter count and FLOPs are therefore not interchangeable. The independent little model is also optimized as if it should classify the full data distribution, while in a cascade it mainly needs to classify easy, high-confidence inputs and defer harder cases. The joint search sees this routing behaviour during evaluation: it directly optimizes expected cascade FLOPs and cascade accuracy, where the little-model cost is always paid and wrong confident exits are harmful. This encourages cheaper little branches that act more like cascade front-ends rather than standalone classifiers. This partly explains why the independent baseline can have a high exit ratio while still producing a weaker accuracy-FLOPs trade-off.

B. Effect of Retraining Loss

The C1-C4 ablation shows that the retraining objective changes how the selected cascades route inputs, but the architecture selected by NAS remains a major factor in the final accuracy-FLOPs trade-off. CE remains a strong baseline on CIFAR-10. The cascade-aware loss without smoothing, C3, slightly improves cascade accuracy and lowers expected FLOPs compared with C1, but the difference is small. The clearer effect comes from label smoothing: C2 and C4 reduce harmful routing errors and obtain higher cascade accuracy by deferring more samples to the big model. This makes the result useful even though it is not a simple standalone win for the custom loss: the architecture selection, operating threshold, confidence behaviour, and smoothing all shape the final cascade.

Figure 6 should therefore be read as evidence about operating behaviour rather than as a clean replacement of one

loss by another. The cascade-aware variants are competitive, and C3 moves in the desired direction by achieving marginally higher accuracy with slightly lower FLOPs than C1. However, the difference between C1 and C3 is small compared with the effect of smoothing in C2 and C4. The current CIFAR-10 setting therefore does not prove that the cascade-aware terms are the dominant source of improvement, although the loss remains useful as a routing-aware regularizer and a plausible direction for further tuning.

C. Routing and Confidence

The experiments also show that confidence routing is central to cascade usefulness. The system is not determined only by little-model and big-model standalone accuracy. It also depends on whether the little model is confident on the right samples. A high exit ratio is beneficial only when the exited samples are mostly correct; otherwise, it creates avoidable routing errors where the cascade exits early even though the big model would have been correct. This is why ECE, routing error rate, and exit ratio are necessary alongside accuracy and FLOPs when interpreting the results. In the final ablation, C4 has the lowest routing error, while C3 has a higher exit ratio and lower FLOPs; this illustrates the practical tension between compute saving and safe early exits.

Figure 7 shows why routing cannot be reduced to a single exit-ratio number. A cascade with more early exits may be cheaper, but it is only useful if those exits are reliable. Conversely, a cascade with strong accuracy may still use more computation if it defers many samples. Routing error captures the specific failure case that matters for the threshold router: the little model exits confidently and incorrectly on a sample that the big model would have classified correctly. The appendix compares the independent baseline’s routing diagnostics with the C1-C4 ablations. This supports a cautious interpretation: co-search does not always produce lower ECE, but the lower-routing joint variants reduce harmful early exits more effectively.

Figure 8 shows why the fixed threshold $\tau = 0.70$ should be interpreted as one operating point rather than a universal best setting. The same trained cascade can move along a spectrum from cheaper and more exit-heavy to more accurate and more deferral-heavy simply by changing the threshold. This is important for deployment, because the preferred threshold depends on whether the target device prioritizes accuracy, latency, or energy. The threshold should also be interpreted probabilistically. If the little model were perfectly calibrated, predictions with confidence around 0.70 would still be expected to be wrong about 30% of the time. A threshold of 0.70 therefore does not guarantee correctness; it defines how much uncertainty the cascade accepts before deferring to the big model. We use $\tau = 0.70$ because it is low enough to allow the little model to exit on a meaningful fraction of inputs under the limited proxy-evaluation budget, but high enough to avoid routing many clearly low-confidence predictions through the little model. Since the joint search also evaluates candidates at this threshold, the selected architectures

are partly optimized for $\tau = 0.70$. This is appropriate for a fixed-threshold deployment setting, but it also means that the fixed-threshold results may be relatively favourable to the threshold used during search.

For C3 and C4, $\tau = 0.70$ also enters the retraining objective itself. The cascade-aware loss uses a soft routing probability based on the little model’s confidence relative to the threshold, so it encourages behaviour that is useful near this operating point. This helps explain the threshold-sweep shape: the cascade-aware variants, especially C3, are competitive at lower and mid thresholds where early exits remain common, but can lose their advantage once the threshold is pushed above 0.70 and the routing policy moves away from the one emphasized during training.

D. Pico 2 Deployment Interpretation

The Pico 2 measurements support the main dynamic-inference assumption behind the analytical FLOPs metric: early exits are cheaper than deferrals. For the Arch2 cascades, samples that exit after the little model take about 40 ms, whereas deferred samples take about 204 ms because the big model is evaluated after the little model. This approximately five-fold per-sample latency difference means that the exit ratio has a direct hardware effect. C1 and C3, which exit on about 43% of samples, have lower average latency than C2 and C4, which defer more often. Thus, the measured latency confirms that avoiding the big model produces the expected compute saving on the Pico 2.

At the same time, the hardware results show why analytical MACs should be treated as a proxy rather than a complete deployment model. Within the Arch2 family, the average analytical FLOPs increase from roughly 0.87M MACs for C1 and C3 to roughly 1.03M MACs for C4, and measured latency increases in the same direction. This agreement is useful because it validates the FLOPs objective as a reasonable ranking signal among closely related cascades. However, the independent baseline breaks any simple FLOPs-to-latency interpretation. Its average cascade cost is about 2.05M MACs, but its measured latency is 425.77 ms, far more than a simple doubling of the Arch2 latency would suggest. The independent little model alone takes about 131.93 ms, more than three times the Arch2 little path, even though both are small enough to fit the search budget.

This mismatch is likely caused by hardware details that are intentionally abstracted away during NAS. The analytical FLOPs calculation counts multiply-accumulate work, but it does not model TensorFlow Lite Micro kernel implementations, depthwise versus standard convolution efficiency on the Pico 2, memory movement, cache effects, tensor layout conversions, or arena allocation pressure. It also does not capture the fact that deployment memory is split between flash-resident model data and SRAM-resident tensor arenas. The Arch2 models used a stable 128/256 KiB little/big arena split, whereas the independent baseline required a much larger big-model arena and several diagnostic builds. This reinforces the distinction made in the methodology: analytical MACs and

estimated memory are useful for search and comparison, but final deployment still requires hardware measurement.

Overall, the Pico 2 results strengthen the interpretation of RQ1 and RQ3. The joint Arch2 cascades do not only look better on the analytical accuracy–FLOPs frontier; they also run substantially faster on the target microcontroller. The measured route split explains much of the latency difference within a fixed architecture, while the independent baseline shows that a high exit ratio is not sufficient when the little model itself is slow and memory-hungry. The deployment results therefore support the expected compute savings of threshold-based cascades, but they also show that FLOPs, exit ratio, and memory must be interpreted together rather than as independent deployment predictors.

E. Dataset and Framework Interpretation

The dataset also affects the interpretation. CIFAR-10 is small enough to make the full NAS and retraining pipeline feasible, but it may not create a strong division between easy examples for the little model and hard examples for the big model. In several runs, the little model approaches the big model’s standalone accuracy. This reduces the natural room for a cascade-aware loss to specialize the two models. A more complex dataset could make the big/little division of labour clearer, but would also increase the cost of NAS.

Overall, the contributions are the joint NSGA-II co-search setup, the cascade-aware co-training loss, the C1-C4 retraining ablation, and the analysis of how threshold choice, exit ratio, routing errors, and capacity allocation shape the final accuracy–FLOPs trade-off. The results show that this framework can find memory-feasible dynamic cascades and that joint co-search is more effective than independently searching the two models at finding strong cascade operating points. The analysis further exposes why different operating points trade accuracy against expected compute.

F. Limitations

The main limitations concern the gap between a feasible NAS experiment and the ideal exhaustive optimization of a deployed MCU cascade.

1) *Proxy Training*: The architecture search relies on proxy training to evaluate candidate cascades. This is necessary because a full retraining run for every little-big pair would make the search computationally impractical. However, proxy training also means that each candidate is judged before it has necessarily reached its full potential. Some architectures may learn slowly and therefore appear weak during the proxy stage, while others may look strong early but not retain that advantage after longer training. This limits how confidently the search ranking can be interpreted. The final retraining step reduces this issue for selected architectures, but it does not recover candidates that were discarded because their proxy performance was misleading.

2) *Dataset Size*: The experiments use CIFAR-10 with a train/validation split. CIFAR-10 is useful for this project because it is small enough to support repeated NAS, retraining,

threshold evaluation, export, and microcontroller testing. At the same time, the dataset is limited in both size and visual complexity. The learned cascade behaviour may therefore depend on CIFAR-10-specific properties, such as the distribution of easy and difficult samples, the class set, and the image resolution. A larger or more complex dataset could change which architectures are feasible, which loss functions are useful, and how often the little model can safely handle an example without deferring to the big model.

3) *Search Space Size*: The search space contains a very large number of possible cascade configurations. Each cascade is defined not only by one architecture, but by a little model, a big model, and how their capacities interact under the routing rule. Even with a restricted CNN design space, the number of possible combinations grows quickly as block depth, channel count, kernel size, convolution type, and residual choices are varied. With relatively few hard bounds, the search must explore a space containing many infeasible or unpromising candidates before it finds architectures that satisfy the Pico 2 constraints and still perform well. This can make the search inefficient and increases the chance that good feasible regions are only sparsely sampled. More role-specific constraints, stronger feasibility filters, or hardware-informed priors could make the search more focused.

4) *Search and Training Budget*: The NAS experiments use a finite search and training budget. The joint search uses a population of 20 for 20 generations, while the independent baseline uses 20 individuals for 10 generations per role. Each candidate is proxy-trained for only 5 epochs. These settings make the experiment feasible, but they also mean that the search only samples a small fraction of the possible architecture space. The independent setup also matches the approximate number of proxy-trained candidates rather than giving each role the same 20-generation evolutionary depth as the joint search. This was a practical compromise because the independent baseline performs two sequential searches, but a stricter comparison would be to increase the generation budget for both methods. Increasing the number of generations, population size, or proxy-training epochs could improve the reliability of the selected architectures, but would also increase runtime substantially. The same applies to final retraining: 50 epochs is long enough to obtain useful comparisons across many variants, but many CIFAR-style training setups use longer schedules such as 100-200 epochs. A longer retraining schedule could improve absolute accuracy and might give the cascade-aware loss more time to influence confidence and routing behaviour. The reported results should therefore be understood as the outcome of a practical NAS budget rather than an exhaustive optimum.

5) *Float32 Deployment*: The deployment experiment uses float32 TensorFlow Lite Micro models instead of quantized models. This choice makes the export path easier to validate because the deployed models remain close to the PyTorch models used during training. It also avoids adding quantization error as a separate variable in the first deployment experiment. However, float32 inference is not the most memory-efficient

representation for an MCU. Compared with int8 quantization, float32 weights require substantially more memory, reducing the amount of model capacity available within the same Pico 2 budget. This matters directly for the project because the search is trying to allocate capacity between the little and big models. A quantized deployment could allow larger architectures, lower latency, or different routing behaviour, but it could also change confidence values and therefore affect the cascade’s exit decisions.

6) *Pico Deployment Measurements*: The Pico 2 evaluation measures on-device latency and routing behaviour, which is an important step beyond reporting only proxy FLOPs. The route-aware energy test adds an initial direct measurement of energy on a 1,000-sample embedded subset, but it is less comprehensive than the 10,000-sample serial latency evaluation and is reported only for the final joint-vs-independent comparison. For microcontroller applications, especially battery-powered ones, energy may be as important as latency. A cascade that is faster is not necessarily the most energy-efficient if its power draw differs across model paths or if repeated big model invocations have a disproportionate cost. Future evaluation should therefore include repeated energy measurements across all C1–C4 variants, additional boards, and quantized models in addition to accuracy, route split, and latency.

G. Future Work

Future work should first evaluate the framework on a more complex dataset such as CIFAR-100 or Tiny ImageNet. This would test whether the cascade-aware design becomes more useful when there is a clearer separation between easy and hard inputs. A stronger dataset would also make it easier to judge whether the big model provides a meaningful advantage over the little model.

A second direction is to repeat the comparison with a larger search and training budget. The current setup matches the approximate number of proxy-trained candidates between joint and independent NAS, but independent NAS uses 10 generations per role while joint NAS evolves complete pairs for 20 generations. Future experiments should test larger population sizes, generation counts, and proxy-training budgets for both methods to evaluate whether the observed advantage remains under a larger search budget. The selected architectures should also be retrained for longer schedules, such as 100-200 epochs, to estimate final accuracy more precisely and to test whether cascade-aware losses need more training time to shape confidence and routing behaviour.

A further direction is to improve the routing mechanism. This work uses maximum softmax confidence because it is simple, cheap, and directly compatible with an early-exit cascade. Other routing rules could be tested, including margin-based routing, entropy-based routing, or a small learned router. These alternatives may identify uncertain samples more reliably than raw confidence.

The cascade-aware loss should also be explored more systematically. The current weights were chosen from a small set of manual configuration tests, where the setting with a

higher defer-cost term showed the most potential for C3. This is not enough to claim that the setting is globally optimal. A broader multi-seed sweep over defer cost, confident-wrong penalty, routing sharpness, auxiliary loss weights, and training threshold could show whether the loss can more consistently improve cascade accuracy or exit ratio. It would also be useful to compare training at the same threshold used for final evaluation against training with a range of thresholds.

Finally, future experiments should include stronger deployment-oriented baselines and measurements. Single little-only and big-only models, cascades matched by FLOPs, and quantized int8 variants would make the accuracy-compute trade-off easier to interpret. Hardware evaluation should also include latency, peak memory, and energy measurements, since FLOPs and parameter count are only proxies for embedded deployment cost.

VII. CONCLUSION

This work investigated NAS-driven big/little cascades for input-adaptive inference under a small deployment-memory budget. The proposed framework combines joint cascade architecture search, constrained model selection, retraining ablations, and threshold-based evaluation. The results show that joint co-search can find substantially better cascades than an independently searched baseline. At the main threshold $\tau = 0.70$, the joint variants cover a better accuracy-FLOPs region: C3 is the cheaper cascade-aware operating point, while C4 is the higher-accuracy, higher-compute operating point. Both support the central argument that the little model, big model, and routing rule should be designed as a coupled system.

The retraining ablation shows a more nuanced result. The custom cascade-aware loss is not a dramatic standalone replacement for CE on CIFAR-10, but the C3 setting slightly improves accuracy and lowers FLOPs compared with CE. Combining cascade-aware loss with label smoothing gives the highest average accuracy and the lowest routing error among the C1-C4 variants, but it also uses more FLOPs. This suggests that cascade-aware objectives are useful, but their benefit depends on how they interact with confidence calibration, exit ratio, and the selected architecture.

The experiments also show that threshold choice, exit ratio, calibration, and routing error are essential for interpreting cascade systems. A cascade is only efficient when the little model exits on samples it can classify correctly; a high exit ratio alone is not sufficient. The Pico 2 deployment confirms this hardware view: the joint Arch2 cascade has much lower measured latency and lower route-aware energy than the independent baseline, even though the independent baseline exits more often. This shows why analytical MACs are useful search proxies but cannot fully capture TensorFlow Lite Micro latency, arena pressure, and board-level energy behaviour. Overall, the project demonstrates a reproducible framework for NAS-driven dynamic inference and shows that joint cascade design can improve the accuracy-compute trade-off, while leaving open important questions about loss design,

calibration, larger datasets, quantization, and hardware-aware optimization.

ACKNOWLEDGMENTS

The authors would like to thank Emil Jorgensen and Francesco Daghero for their supervision, feedback, and guidance throughout the project. We also thank the University of Southern Denmark for providing access to computing resources and development facilities used during the experiments.

APPENDIX
ADDITIONAL DIAGNOSTIC PLOTS

This appendix collects supporting plots that are useful for interpreting the main results, but are not necessary for the primary RQ1-RQ3 narrative. The figures should be read as diagnostic evidence rather than additional headline results.

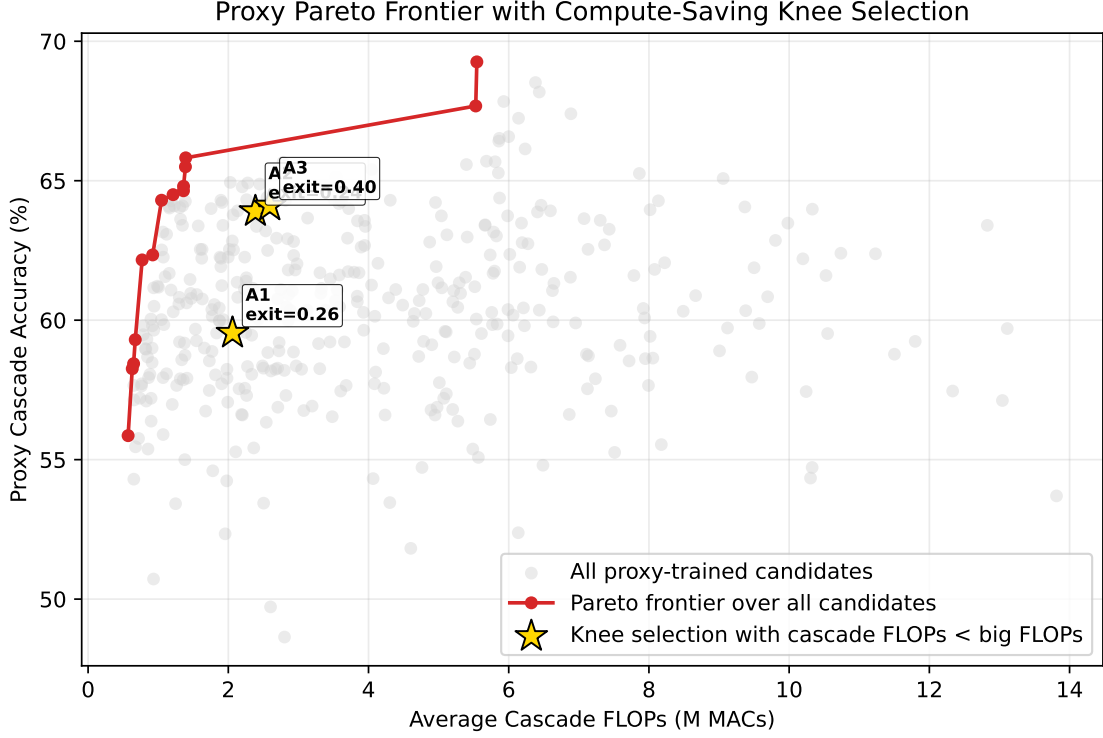


Fig. 10. Proxy Pareto frontier used to select joint architectures for full retraining. The selected candidates are chosen from the compute-saving knee region after feasibility checks and the constraint $F_{\text{cascade}} < F_b$, not simply by taking the three highest proxy-accuracy points. The figure therefore documents the selection heuristic; final claims are based on fully retrained test-set results rather than the 5-epoch proxy scores.

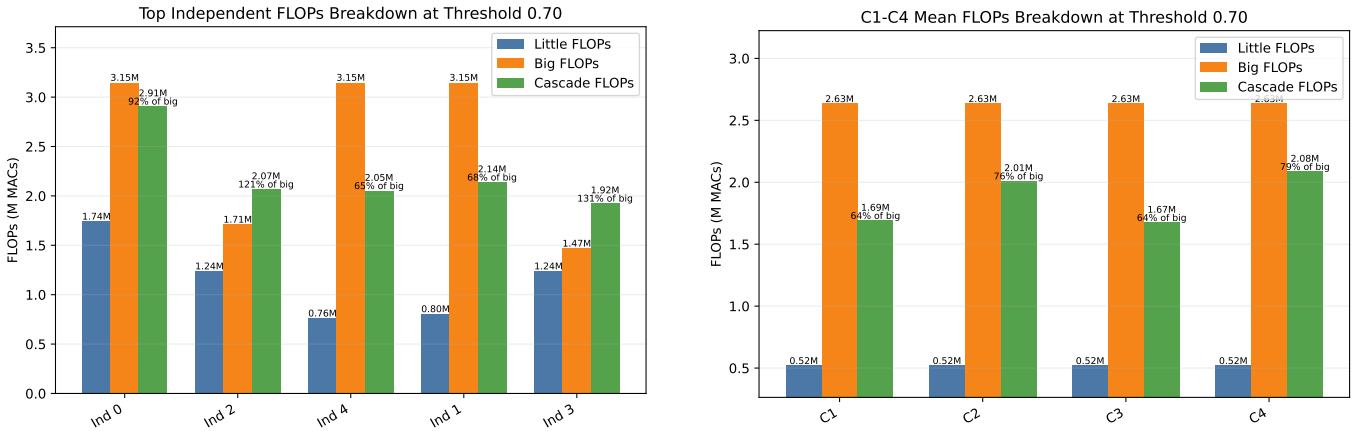


Fig. 11. FLOPs breakdown at $\tau = 0.70$. The independent baseline and joint variants use different selected architecture pairs, while the C1-C4 joint ablations use the same selected architectures. Changes in expected cascade FLOPs within the joint ablation therefore mainly reflect differences in exit ratio caused by the retraining objective.

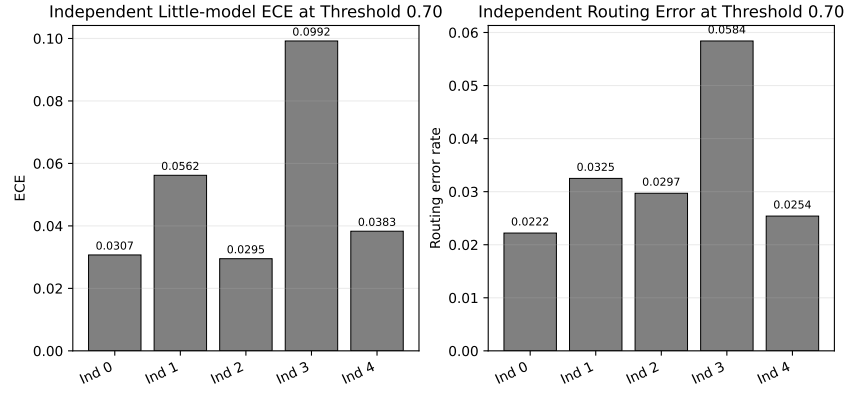


Fig. 12. Independent-baseline routing diagnostics at $\tau = 0.70$. The independent baseline can have lower ECE than some joint variants, but it still has lower cascade accuracy, showing that calibration alone is not sufficient to explain the final trade-off.

Confidence vs Accuracy Reliability Diagrams

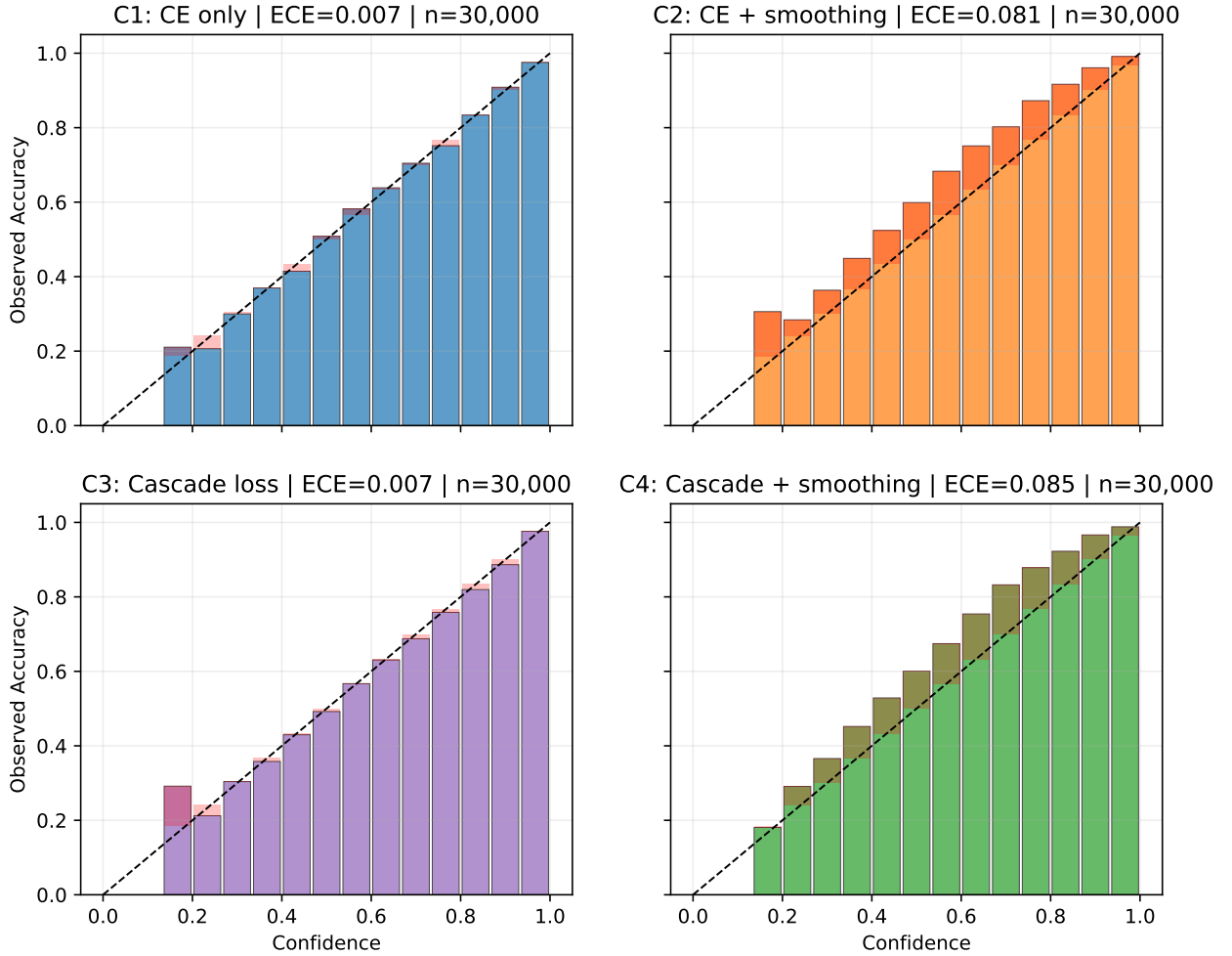


Fig. 13. Confidence-accuracy reliability diagrams for the C1-C4 retraining variants. The figure complements the ECE and routing-error summaries by showing where each variant is under- or over-confident across confidence bins.

REFERENCES

- [1] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, pp. 1–21, 2019.
- [2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [3] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, and W. Banzhaf, “NSGA-Net: Neural architecture search using multi-objective genetic algorithm,” in *Genetic and Evolutionary Computation Conference (GECCO)*, 2019.
- [4] A. Chebykin, E. Kijak, J. Hanna, and T. Asselborn, “ENCAS: A general framework for efficient nas of model cascades,” in *Genetic and Evolutionary Computation Conference (GECCO)*, 2022.
- [5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *International Conference on Machine Learning (ICML)*, 2017.
- [6] R. Müller, S. Kornblith, and G. Hinton, “When does label smoothing help?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] L. Li, Y. Luo, X. Qiao, H. Chen, and Y. Zhang, “Learning to cascade: Confidence calibration for improving the accuracy and computational cost of cascade inference systems,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [8] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, “Dynamic neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2021.
- [9] E. Park, D. Kim, S. Kim, Y.-D. Kim, G. Kim, S. Yoon, and S. Yoo, “Big/little deep neural network for ultra low power inference,” in *International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2015.
- [10] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” *arXiv preprint arXiv:1709.01686*, 2017.
- [11] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, “Multi-scale dense networks for resource efficient image classification,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [12] Y. Kaya, S. Hong, and T. Dumitras, “Shallow-deep networks: Understanding and mitigating network overthinking,” in *International Conference on Machine Learning (ICML)*, 2019.
- [13] M. Gambella and M. Roveri, “EDANAS: Adaptive neural architecture search for early exit neural networks,” in *International Joint Conference on Neural Networks (IJCNN)*, 2023.
- [14] M. Gambella, J. Pomponi, S. Scardapane, and M. Roveri, “NACHOS: Neural architecture search for hardware constrained early exit neural networks,” *arXiv preprint arXiv:2401.13330*, 2025.
- [15] F. Regol, A. Dhurandhar, B. Mehta, S. Ravi, J. Bian, and A. Hassidim, “Jointly-learned exit and inference for a dynamic neural network: JEI-DNN,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [16] J. Mukhoti, V. Kulharia, A. Sanber, K. Mayfield, P. H. Torr, and P. Sturgess, “Calibrating deep neural networks using focal loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 15 288–15 299.
- [17] H. Benmeziane, K. El Maghraoui, H. Ouarnoughi, S. Niar, M. Wistuba, and N. Wang, “A comprehensive survey on hardware-aware neural architecture search,” *arXiv preprint arXiv:2101.09336*, 2021.
- [18] T.-J. Yang, A. Howard, B. Chen, X. Zhang, A. Go, M. Sandler, V. Sze, and H. Adam, “NetAdapt: Platform-aware neural network adaptation for mobile devices,” in *European Conference on Computer Vision (ECCV)*, 2018.
- [19] H. Cai, L. Zhu, and S. Han, “ProxylessNAS: Direct neural architecture search on target task and hardware,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [20] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [21] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.